

Final Project: The Curse of Dimensionality

Final Project

December 12, 2025

Project Overview

This is a competitive project where you'll compare Johnson-Lindenstrauss (JL) random projection vs PCA for dimensionality reduction on CIFAR-10 (Cats vs Dogs), then classify images using **Binary Logistic Regression**.

Your goal: Achieve the highest test accuracy while exploring the trade-offs between methods.

Dataset: CIFAR-10 Binary Classification

We'll use a subset of CIFAR-10 focusing on binary classification:

- **Two classes only:** Cat (class 3) vs Dog (class 5)
- Approximately 10,000 images (5,000 cats + 5,000 dogs) in training set
- Approximately 2,000 images (1,000 cats + 1,000 dogs) in test set
- Image size: $32 \times 32 \times 3 = 3,072$ dimensions
- Each image is flattened to a 3,072-dimensional vector
- Labels: 0 = Cat, 1 = Dog

Competition: Best Accuracy Wins!

- You can use **any combination** of JL and PCA
- You can use **any target dimensions** k (e.g., 50, 100, 200, 500, 1000, 2000)
- You can tune Binary Logistic Regression hyperparameters
- Report your **best test accuracy** on the full test set
- Must use standard Binary Logistic Regression (no neural networks or ensemble methods)

1 Part 1: Implementation

Implement both methods from scratch using only NumPy:

1.1 Task 1.1: Johnson-Lindenstrauss Random Projection

```

1 def jl_random_projection(X_train, X_test, n_components, random_state=None):
2     """
3     Apply Johnson-Lindenstrauss random projection
4
5     Parameters:
6     -----
7     X_train : ndarray, shape (n_train, d)    Training data
8     X_test  : ndarray, shape (n_test, d)     Test data
9     n_components : int                      Target dimension k
10    random_state : int                      Random seed
11
12    Returns:
13    -----
14    X_train_projected : ndarray, shape (n_train, k)
15    X_test_projected  : ndarray, shape (n_test, k)
16    projection_matrix : ndarray, shape (k, d)
17
18    Algorithm:
19    -----
20    1. Set random seed if provided
21    2. Generate projection matrix  $R \sim N(0, 1/k)$  with shape (k, d)
22    3. Project:  $X_{\text{projected}} = X @ R.T$ 
23    """
24    # Your implementation here
25    pass

```

1.2 Task 1.2: Principal Component Analysis (PCA)

```

1 def pca(X_train, X_test, n_components):
2     """
3     Apply Principal Component Analysis
4
5     Parameters:
6     -----
7     X_train : ndarray, shape (n_train, d)    Training data
8     X_test  : ndarray, shape (n_test, d)     Test data
9     n_components : int                      Number of pc
10
11    Returns:
12    -----
13    X_train_pca : ndarray, shape (n_train, k)
14    X_test_pca  : ndarray, shape (n_test, k)
15    components  : ndarray, shape (k, d)       Principal components
16    explained_variance : ndarray, shape (k,)   Variance explained
17    by each PC
18    mean : ndarray, shape (d,)                Mean of training
19    data
20
21    Algorithm:
22    -----

```

```

21     1. Center training data: X_centered = X_train - mean(X_train)
22     2. Compute covariance matrix: Cov = X_centered.T @ X_centered / (n
23         -1)
24     3. Compute eigenvalues and eigenvectors of covariance matrix
25     4. Sort eigenvectors by eigenvalues (descending order)
26     5. Select top k eigenvectors as principal components
27     6. Project training data: X_train_pca = X_centered @ components.T
28     7. Center and project test data using training mean
29     """
30     # Your implementation here
    pass

```

1.3 Task 1.3: Binary Logistic Regression

Implement binary logistic regression from scratch:

```

1  def sigmoid(z):
2      """Sigmoid function"""
3      return 1 / (1 + np.exp(-z))
4
5  def binary_logistic_regression(X_train, y_train, X_test, y_test,
6      learning_rate=0.01, n_iterations=1000,
7      regularization=0.0):
8      """
9      Binary Logistic Regression using gradient descent
10
11      Parameters:
12      -----
13      X_train : ndarray, shape (n_train, d)    Training features
14      y_train : ndarray, shape (n_train,)      Training labels (0 or 1)
15      X_test  : ndarray, shape (n_test, d)     Test features
16      y_test  : ndarray, shape (n_test,)       Test labels (0 or 1)
17      learning_rate : float                    Learning rate for gradient
18          descent
19      n_iterations : int                       Number of iterations
20      regularization : float                  L2 regularization parameter
21          (lambda)
22
23      Returns:
24      -----
25      weights : ndarray, shape (d,)            Learned weights
26      bias : float                             Learned bias term
27      train_accuracy : float                   Training accuracy
28      test_accuracy : float                    Test accuracy
29
30      Algorithm:
31      -----
32      1. Initialize weights w = 0, bias b = 0
33      2. For each iteration:
34          a. Compute predictions: y_pred = sigmoid(X @ w + b)
35          b. Compute gradient: dw = X.T @ (y_pred - y) / n + lambda * w
36          c. Compute bias gradient: db = mean(y_pred - y)
37          d. Update: w = w - learning_rate * dw
38          e. Update: b = b - learning_rate * db
39      3. Compute final accuracy on train and test sets
40      """
41      # Your implementation here
    pass

```

1.4 Task 1.4: Verify Your Implementation

Compare your implementations with scikit-learn:

```

1 import numpy as np
2 from sklearn.decomposition import PCA as SklearnPCA
3 from sklearn.random_projection import GaussianRandomProjection
4 from sklearn.linear_model import LogisticRegression
5
6 # Test with small sample
7 X_small_train = X_train[:1000]
8 X_small_test = X_test[:200]
9 y_small_train = y_train[:1000]
10 y_small_test = y_test[:200]
11
12 # Test your PCA implementation
13 X_train_yours, X_test_yours, components, var, mean = pca(
14 X_small_train, X_small_test, n_components=100)
15
16 # Compare with sklearn
17 sklearn_pca = SklearnPCA(n_components=100)
18 X_train_sklearn = sklearn_pca.fit_transform(X_small_train)
19 X_test_sklearn = sklearn_pca.transform(X_small_test)
20
21 # Check similarity (should be close, possibly sign-flipped)
22 corr_train = np.abs(np.corrcoef(X_train_yours.flatten(),
23 X_train_sklearn.flatten())[0,1])
24 corr_test = np.abs(np.corrcoef(X_test_yours.flatten(),
25 X_test_sklearn.flatten())[0,1])
26
27 print(f"PCA Train correlation: {corr_train:.4f}") # Should be > 0.99
28 print(f"PCA Test correlation: {corr_test:.4f}") # Should be > 0.99
29
30 # Test JL random projection
31 X_train_jl, X_test_jl, R = jl_random_projection(
32 X_small_train, X_small_test, n_components=100, random_state=42)
33
34 sklearn_jl = GaussianRandomProjection(n_components=100, random_state=42)
35 X_train_sklearn_jl = sklearn_jl.fit_transform(X_small_train)
36 X_test_sklearn_jl = sklearn_jl.transform(X_small_test)
37
38 print(f"JL projection matrix shape: {R.shape}") # Should be (100, 3072)
39
40 # Test Binary Logistic Regression
41 w, b, train_acc, test_acc = binary_logistic_regression(
42 X_small_train, y_small_train, X_small_test, y_small_test,
43 learning_rate=0.1, n_iterations=1000)
44
45 # Compare with sklearn
46 clf_sklearn = LogisticRegression(max_iter=1000, random_state=42)
47 clf_sklearn.fit(X_small_train, y_small_train)
48 sklearn_test_acc = clf_sklearn.score(X_small_test, y_small_test)
49
50 print(f"Your test accuracy: {test_acc:.4f}")
51 print(f"Sklearn test accuracy: {sklearn_test_acc:.4f}")

```

2 Part 2: Systematic Comparison

2.1 Task 2.1: Experiment with Different Dimensions

Test both methods with various target dimensions and plot accuracy vs dimension.

Dimensions to test: $k \in \{50, 100, 200, 500, 1000, 2000\}$

For each method (JL and PCA) and each k :

1. Apply dimensionality reduction:
2. Train Binary Logistic Regression:

2.2 Task 2.2: Create Comparison Plots

Generate the following plots:

1. **Accuracy vs Dimension:** Plot test accuracy vs k for both JL and PCA
2. **Training Time vs Dimension:** Compare computational efficiency
3. **Train vs Test Accuracy:** Check for overfitting across dimensions

3 Part 3: Advanced Analysis

3.1 Task 3.1: Variance Analysis (PCA Only)

For PCA:

- Calculate cumulative explained variance for $k \in \{50, 100, 200, 500, 1000, 2000\}$
- Plot: Cumulative variance vs k
- Find: What k preserves 90%, 95%, 99% variance?

3.2 Task 3.2: Distance Preservation Analysis

For both JL and PCA:

- Sample 500 random image pairs
- Compute original distances: $d_{\text{orig}} = \|x_i - x_j\|_2$
- Compute projected distances: $d_{\text{proj}} = \|f(x_i) - f(x_j)\|_2$
- Calculate relative distortion: $\frac{|d_{\text{proj}} - d_{\text{orig}}|}{d_{\text{orig}}}$
- Plot: Mean distortion vs k (both methods)

4 Tips for Competition

4.1 Strategy 1: Hyperparameter Tuning

Try different settings for your binary logistic regression:

```

1 # Grid search over hyperparameters
2 learning_rates = [0.001, 0.01, 0.1, 1.0]
3 regularizations = [0.0, 0.001, 0.01, 0.1, 1.0]
4 n_iterations_list = [500, 1000, 2000]
5
6 best_accuracy = 0
7 best_params = {}
8
9 for lr in learning_rates:
10 for reg in regularizations:
11 for n_iter in n_iterations_list:
12 w, b, _, test_acc = binary_logistic_regression(
13 X_train_reduced, y_train, X_test_reduced, y_test,
14 learning_rate=lr, n_iterations=n_iter,
15 regularization=reg)
16
17 if test_acc > best_accuracy:
18 best_accuracy = test_acc
19 best_params = {'lr': lr, 'reg': reg, 'n_iter': n_iter}
20
21 print(f"Best accuracy: {best_accuracy:.4f}")
22 print(f"Best parameters: {best_params}")

```

4.2 Strategy 2: Data Preprocessing

```

1 # Standardization (after dimensionality reduction)
2 from sklearn.preprocessing import StandardScaler
3
4 scaler = StandardScaler()
5 X_train_scaled = scaler.fit_transform(X_train_reduced)
6 X_test_scaled = scaler.transform(X_test_reduced)
7
8 # This can significantly improve convergence!

```

4.3 Strategy 3: Optimal k Selection

- PCA: Check which k preserves 95–99% variance
- JL: Try k values near theoretical bound: $k \geq \frac{4 \log n}{\epsilon^2}$
- Test hypothesis: More dimensions isn't always better (overfitting!)

4.4 Strategy 4: Feature Combination

Train on concatenated PCA and JL features:

```

1 # Use different k values for PCA and JL
2 X_train_pca, X_test_pca, _, _, _ = pca(X_train, X_test, n_components=100)
3 X_train_jl, X_test_jl, _ = jl_random_projection(
4 X_train, X_test, n_components=50, random_state=42)
5

```

```

6 # Concatenate features
7 X_train_combined = np.hstack([X_train_pca, X_train_jl])
8 X_test_combined = np.hstack([X_test_pca, X_test_jl])
9
10 # Train on combined features
11 w, b, train_acc, test_acc = binary_logistic_regression(
12 X_train_combined, y_train, X_test_combined, y_test)

```

4.5 Strategy 5: Multiple Random Seeds for JL

Since JL uses random projections, results can vary:

5 Load and Filter CIFAR-10

You can use the following code to load CIFAR-10 and filter required classes.

```

1 import numpy as np
2 from tensorflow.keras.datasets import cifar10
3
4 # Load data
5 (X_train_full, y_train_full), (X_test_full, y_test_full) = cifar10.
   load_data()
6
7 # Filter for cats (3) and dogs (5) only
8 train_mask = (y_train_full.flatten() == 3) | (y_train_full.flatten() == 5)
9 test_mask = (y_test_full.flatten() == 3) | (y_test_full.flatten() == 5)
10
11 X_train = X_train_full[train_mask]
12 y_train = y_train_full[train_mask].flatten()
13 X_test = X_test_full[test_mask]
14 y_test = y_test_full[test_mask].flatten()
15
16 # Convert labels: 3 (cat) -> 0, 5 (dog) -> 1
17 y_train = (y_train == 5).astype(int)
18 y_test = (y_test == 5).astype(int)
19
20 # Flatten images: (n, 32, 32, 3) -> (n, 3072)
21 X_train = X_train.reshape(X_train.shape[0], -1).astype('float32') / 255.0
22 X_test = X_test.reshape(X_test.shape[0], -1).astype('float32') / 255.0
23
24 print(f"Training set: {X_train.shape}") # (~10000, 3072)
25 print(f"Test set: {X_test.shape}") # (~2000, 3072)
26 print(f"Class distribution (train): Cat={np.sum(y_train==0)}, Dog={np.sum(
   y_train==1)}")
27 print(f"Class distribution (test): Cat={np.sum(y_test==0)}, Dog={np.sum(
   y_test==1)}")

```